ONE-DAY IMPLEMENTATION GUIDE

DINO-Based Concept-Grounded Parts Discovery

Migrating from Slot Attention → DINO + KMeans + Concept Classifier

8 Steps · ~6.5 Hours Total · Works on M4 Mac (MPS)

Day Plan Overview

Complete all 8 steps in sequence. Each step ends with a validation command — do not proceed to the next step until validation passes.

#	Step	Files Changed	Est. Time	Cumulative
1	Project setup + dependencies	`requirements.txt, configs/unified_config.yaml`	20 min	0:20
2	DINO extractor module	`src/models/dino_extractor.py [NEW]`	40 min	1:00
3	Feature extraction pipeline	`experiments/extract_dino_features.py [NEW]`	40 min	1:40
4	Patch clusterer + part maps	`src/parts/patch_clusterer.py [NEW]`	50 min	2:30
5	Human labeling tool	`labeling/label_tool.py [NEW]`	50 min	3:20
6	Concept vector builder	`src/concepts/concept_builder.py [NEW]`	40 min	4:00
7	Concept classifier + explanations	`src/classification/concept_classifier.py [NEW]`	60 min	5:00
8	End-to-end runner + MLflow/W&B logging	`experiments/run_dino_pipeline.py [NEW]`	60 min	6:00

What you are KEEPING from existing codebase

✅ data/ folder — all your existing cat images, exactly as-is
✅ configs/unified_config.yaml — we ADD a dino: section, keep everything else
✅ W&B / MLflow integration — plug into new runner in Step 8
✅ notebooks/ — keep for exploration
✅ src/clustering/streamlit_labeler.py — we replace this with a better version in Step 5

What you are ARCHIVING (do not delete — just do not run)

🗄️ src/training/part_trainer.py — Slot Attention trainer (archive, do not delete)
🗄️ src/models/slot_attention.py — Slot Attention model (archive, do not delete)

🗄️ experiments/train_part_discovery.py — old training entrypoint (archive)
🗄️ experiments/extract_parts.py — old extraction (archive)
Tip: git commit everything NOW before making any changes, so you have a clean rollback point.

Step 1 — Project Setup & Dependencies

STEP 1	**Project Setup & Dependencies** MODIFY requirements.txt · ADD dino section to config · CREATE cache/ dir	⏱ Est. Time **20 min**

1a — Git commit first

```
# In your project root — save a clean rollback point
git add -A && git commit -m 'chore: archive slot attention before dino migration'
```

1b — Update requirements.txt

ADD these lines to the bottom of your existing requirements.txt. Do not remove any existing lines.

```
# ── DINO Pipeline additions ──────────────────────────────
torch>=2.0.0
torchvision>=0.15.0
scikit-learn>=1.3.0
ipywidgets>=8.0.0
matplotlib>=3.7.0
seaborn>=0.12.0
Pillow>=9.0.0
tqdm>=4.65.0
# streamlit already in your requirements — no change needed
```

Then run:

```
pip install -r requirements.txt --break-system-packages
```

1c — ADD dino section to configs/unified_config.yaml

Open configs/unified_config.yaml. ADD the following block at the bottom. Do NOT remove anything existing.

```
# ── ADD THIS ENTIRE BLOCK TO BOTTOM OF unified_config.yaml ──

dino:
  model: dino_vits8            # small ViT, best spatial resolution
  image_size: 224
  patch_size: 8                # 28x28 = 784 patches per image
  device: mps                  # M4 Mac
  extract_batch_size: 32
  pin_memory: false            # MUST be false for MPS
  cache_dir: cache/
  features_cache: cache/dino_features.pt
  force_recompute: false

clustering:
  n_clusters: 8                # start here — tune after first visual check
  n_init: 10
  max_iter: 300
  batch_size: 4096             # for MiniBatchKMeans
  random_seed: 42

concepts:
  labels_path: cache/labels.json
  vectors_cache: cache/concept_vectors.pt
  scores_cache: cache/concept_scores.pt
```

```
classification:
  C: 1.0                        # logistic regression regularisation
  max_iter: 1000
  test_size: 0.2
  random_seed: 42
```

1d — Create cache directory

```
mkdir -p cache
echo 'cache/*.pt' >> .gitignore
echo 'cache/*.npy' >> .gitignore
```

✅ Validation — Step 1
Run: python -c "import torch; print(torch.__version__); print(torch.backends.mps.is_available())"
Expected: version printed, then True
Run: python -c "import sklearn, ipywidgets, seaborn; print('deps ok')"
Expected: deps ok

Step 2 — DINO Extractor Module

STEP 2	**DINO Extractor Module** CREATE src/models/dino_extractor.py	⏱ Est. Time **40 min**

2a — Create file: src/models/dino_extractor.py

CREATE this file from scratch. This is a completely new file — nothing from existing src/models/ is changed.

```
# src/models/dino_extractor.py
import torch
import torch.nn.functional as F
import torchvision.transforms as T
from PIL import Image
from pathlib import Path
import warnings
warnings.filterwarnings('ignore')


class DinoExtractor:
    """
    Wraps a frozen DINO ViT-S/8 model.
    Extracts patch-level features: shape [784, 384] per 224x224 image.
    784 = 28x28 spatial grid of 8x8 patches.
    384 = embedding dimension for ViT-S.
    """

    MEAN = [0.485, 0.456, 0.406]
    STD  = [0.229, 0.224, 0.225]

    def __init__(self, model_name: str = 'dino_vits8', device: str = 'mps',
                 image_size: int = 224):
        self.device = torch.device(device)
        self.image_size = image_size
        self.patch_size = 8
        self.grid_size  = image_size // self.patch_size   # 28
        self.feat_dim   = 384

        print(f'Loading {model_name} ...')
        self.model = torch.hub.load('facebookresearch/dino:main',
                                    model_name, pretrained=True)
        self.model.eval()
        self.model.to(self.device)
        # freeze — we never train DINO
        for p in self.model.parameters():
            p.requires_grad = False
        print('DINO loaded and frozen.')

        self.transform = T.Compose([
            T.Resize(256),
            T.CenterCrop(self.image_size),
            T.ToTensor(),
            T.Normalize(self.MEAN, self.STD),
        ])

    def load_image(self, path) -> torch.Tensor:
        """Load and transform a single image to tensor [1, 3, H, W]."""
        img = Image.open(path).convert('RGB')
```

```
        return self.transform(img).unsqueeze(0).to(self.device)

    @torch.no_grad()
    def extract_patches(self, image_tensor: torch.Tensor) -> torch.Tensor:
        """
        Args:  image_tensor [B, 3, 224, 224]
        Returns: patch_features [B, 784, 384]
        """
        feats = self.model.get_intermediate_layers(image_tensor, n=1)[0]
        # feats shape: [B, 785, 384] — index 0 is CLS token
        return feats[:, 1:, :]   # drop CLS → [B, 784, 384]

    @torch.no_grad()
    def extract_attention(self, image_tensor: torch.Tensor) -> torch.Tensor:
        """
        Returns attention maps from last layer heads.
        Shape: [B, num_heads, 785, 785]
        Useful for visualisation and validation.
        """
        return self.model.get_last_selfattention(image_tensor)

    def extract_from_path(self, image_path) -> torch.Tensor:
        """Convenience: load image from disk and extract patches."""
        tensor = self.load_image(image_path)
        return self.extract_patches(tensor).squeeze(0).cpu()  # [784, 384]

    def get_spatial_grid(self):
        """Returns (grid_size, grid_size) = (28, 28)."""
        return self.grid_size, self.grid_size
```

2b — Quick sanity test (run immediately after creating file)

```
# Run this from your project root to validate Step 2
python - <<'EOF'
from src.models.dino_extractor import DinoExtractor
import os

extractor = DinoExtractor(device='mps')

# Use any image from your data folder
test_images = []
for root, _, files in os.walk('data/'):
    for f in files:
        if f.lower().endswith(('.jpg','.jpeg','.png')):
            test_images.append(os.path.join(root, f))
            break
    if test_images: break

feats = extractor.extract_from_path(test_images[0])
print(f'Feature shape: {feats.shape}')   # expect torch.Size([784, 384])
print(f'Feature norm:  {feats.norm(dim=-1).mean():.3f}')  # expect ~1.0
EOF
```

✅ **Validation — Step 2**

Expected output:

```
Loading dino_vits8 ...
DINO loaded and frozen.
```

```
  Feature shape: torch.Size([784, 384])
  Feature norm:  ~1.000
If you see shape [785, 384] — the CLS token was not dropped, re-check feats[:, 1:, :]
If MPS errors occur — change device='cpu' in test, will still work (slower)
```

Step 3 — Feature Extraction Pipeline

STEP 3	**Feature Extraction Pipeline** CREATE experiments/extract_dino_features.py · Results cached to disk	⏱ Est. Time **40 min**

This script does the one-time expensive work: iterates all images, extracts DINO patch features, saves to cache. Once done, every downstream step reads from cache — no re-extraction needed.

3a — Create file: experiments/extract_dino_features.py

```
# experiments/extract_dino_features.py
"""
One-time DINO feature extraction for all images in data/.
Saves to cache/dino_features.pt — all subsequent steps load from here.
Runtime: ~15-20 min for 1000 images on M4 Mac.
"""
import sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

import torch
from pathlib import Path
from tqdm import tqdm
import yaml
from src.models.dino_extractor import DinoExtractor


SUPPORTED = {'.jpg', '.jpeg', '.png', '.bmp', '.webp'}

def collect_images(data_dir: str):
    """Walk data_dir, collect all image paths and their class labels."""
    paths, labels, class_names = [], [], []
    data_path = Path(data_dir)
    # Expects structure: data/class_name/image.jpg
    for class_dir in sorted(data_path.iterdir()):
        if not class_dir.is_dir(): continue
        class_name = class_dir.name
        if class_name not in class_names:
            class_names.append(class_name)
        class_id = class_names.index(class_name)
        for img_path in sorted(class_dir.iterdir()):
            if img_path.suffix.lower() in SUPPORTED:
                paths.append(str(img_path))
                labels.append(class_id)
    return paths, labels, class_names


def extract_all(config_path: str = 'configs/unified_config.yaml'):
    with open(config_path) as f:
        cfg = yaml.safe_load(f)

    dcfg = cfg['dino']
    cache_path = dcfg['features_cache']

    if Path(cache_path).exists() and not dcfg.get('force_recompute', False):
        print(f'Cache exists at {cache_path}. Set force_recompute: true to re-run.')
        data = torch.load(cache_path)
```

```
        print(f"Loaded: {data['features'].shape[0]} patches from
{len(data['image_paths'])} images")
        return data

    extractor = DinoExtractor(
        model_name=dcfg['model'],
        device=dcfg['device'],
        image_size=dcfg['image_size']
    )

    # collect images — adapt data_dir to your actual folder
    data_dir = cfg.get('data', {}).get('root', 'data/')
    image_paths, image_labels, class_names = collect_images(data_dir)
    print(f'Found {len(image_paths)} images across classes: {class_names}')

    all_features = []
    all_image_ids = []
    all_patch_ids = []
    all_labels = []
    failed = []

    for img_idx, (img_path, label) in enumerate(tqdm(zip(image_paths, image_labels),
                                                      total=len(image_paths),
                                                      desc='Extracting')):
        try:
            feats = extractor.extract_from_path(img_path)  # [784, 384]
            all_features.append(feats)
            all_image_ids.extend([img_idx] * 784)
            all_patch_ids.extend(list(range(784)))
            all_labels.extend([label] * 784)
        except Exception as e:
            print(f'  Warning: failed on {img_path}: {e}')
            failed.append(img_path)
            continue

    if not all_features:
        raise RuntimeError('No features extracted — check your data/ directory
structure')

    features_tensor = torch.cat(all_features, dim=0)  # [N*784, 384]

    cache = {
        'features':    features_tensor,
        'image_ids':   torch.tensor(all_image_ids, dtype=torch.long),
        'patch_ids':   torch.tensor(all_patch_ids, dtype=torch.long),
        'patch_labels': torch.tensor(all_labels,  dtype=torch.long),
        'image_paths': image_paths,
        'class_names': class_names,
        'image_labels': image_labels,
        'grid_size':   28,
        'feat_dim':    384,
    }

    Path(cache_path).parent.mkdir(parents=True, exist_ok=True)
    torch.save(cache, cache_path)
    print(f'Saved {features_tensor.shape[0]:,} patch features → {cache_path}')
    if failed:
        print(f'Failed images ({len(failed)}): {failed[:5]}')
    return cache
```

```
if __name__ == '__main__':
    extract_all()
```

3b — Run extraction

```
cd /path/to/your/project
python experiments/extract_dino_features.py
# Expected: progress bar, then:
# Saved 784000 patch features → cache/dino_features.pt   (for 1000 images)
```

✅ Validation — Step 3

Run: python -c "import torch; d=torch.load('cache/dino_features.pt'); print(d['features'].shape); print('classes:', d['class_names'])"

Expected: torch.Size([N, 384]) where N = num_images × 784

Expected: classes: ['cat'] (or ['cat', 'dog'] if you have multiple classes)

Cache file should be 200-800 MB depending on image count — this is normal

---

Step 4 — Patch Clusterer & Part Map Visualiser

STEP 4	**Patch Clusterer + Part Maps** CREATE src/parts/patch_clusterer.py · CREATE scripts/validate_clusters.py	⏱ Est. Time **50 min**

4a — Create file: src/parts/patch_clusterer.py

```
# src/parts/patch_clusterer.py
import torch
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from sklearn.cluster import MiniBatchKMeans
from sklearn.preprocessing import normalize
from pathlib import Path
from PIL import Image
import yaml


class PatchClusterer:
    def __init__(self, n_clusters: int = 8, random_seed: int = 42):
        self.n_clusters = n_clusters
        self.kmeans = MiniBatchKMeans(
            n_clusters=n_clusters,
            n_init=10,
            max_iter=300,
            batch_size=4096,
            random_state=random_seed
        )
        self.labels_ = None
        self.centers_ = None

    def fit(self, features: torch.Tensor) -> np.ndarray:
        """
        Args: features [N, 384] — all patch features
        Returns: cluster labels [N]
        """
        print(f'Clustering {features.shape[0]:,} patches into {self.n_clusters}
clusters...')
        X = normalize(features.numpy(), norm='l2')  # L2 normalise before KMeans
        self.kmeans.fit(X)
        self.labels_ = self.kmeans.labels_
        self.centers_ = torch.tensor(self.kmeans.cluster_centers_, dtype=torch.float32)
        print(f'Done. Cluster sizes: {np.bincount(self.labels_)}')
        return self.labels_

    def predict(self, features: torch.Tensor) -> np.ndarray:
        X = normalize(features.numpy(), norm='l2')
        return self.kmeans.predict(X)

    def get_part_map(self, image_idx: int, image_ids: torch.Tensor) -> np.ndarray:
        """
        Returns [28, 28] grid of cluster labels for one image.
        """
        mask = (image_ids == image_idx).numpy()
        patch_labels = self.labels_[mask]   # 784 labels
        return patch_labels.reshape(28, 28)
```

```
    def save(self, path: str):
        import pickle
        with open(path, 'wb') as f:
            pickle.dump(self.kmeans, f)
        torch.save(self.centers_, path.replace('.pkl', '_centers.pt'))
        print(f'Clusterer saved to {path}')

    @classmethod
    def load(cls, path: str, n_clusters: int = 8):
        import pickle
        obj = cls(n_clusters=n_clusters)
        with open(path, 'rb') as f:
            obj.kmeans = pickle.load(f)
        return obj


# ── Visualisation helpers ──────────────────────────────────────

def visualise_part_map(image_path: str, part_map: np.ndarray,
                       n_clusters: int = 8, alpha: float = 0.55,
                       save_path: str = None):
    """
    Overlay a [28,28] part map onto the original image.
    Each cluster gets a distinct colour.
    """
    img = np.array(Image.open(image_path).convert('RGB').resize((224, 224)))
    part_map_full = np.array(Image.fromarray(part_map.astype(np.uint8))
                             .resize((224, 224), Image.NEAREST))

    cmap = cm.get_cmap('tab10', n_clusters)
    overlay = cmap(part_map_full / max(n_clusters - 1, 1))[:, :, :3]
    blended = (1 - alpha) * (img / 255.0) + alpha * overlay
    blended = np.clip(blended, 0, 1)

    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    axes[0].imshow(img);            axes[0].set_title('Original');
axes[0].axis('off')
    axes[1].imshow(part_map_full, cmap='tab10', vmin=0, vmax=n_clusters-1)
                                    axes[1].set_title('Part Map');
axes[1].axis('off')
    axes[2].imshow(blended);        axes[2].set_title('Overlay');
axes[2].axis('off')
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=120, bbox_inches='tight')
        print(f'Saved: {save_path}')
    plt.show()


def visualise_cluster_samples(cluster_id: int, features: torch.Tensor,
                              labels: np.ndarray, image_ids: torch.Tensor,
                              patch_ids: torch.Tensor, image_paths: list,
                              n_samples: int = 9, save_path: str = None):
    """
    Show n_samples image patches from a given cluster.
    This is the input to your human labeling step.
    """
    cluster_mask = (labels == cluster_id)
    idxs = np.where(cluster_mask)[0]
```

```
    if len(idxs) == 0:
        print(f'Cluster {cluster_id} is empty'); return

    sample_idxs = np.random.choice(idxs, min(n_samples, len(idxs)), replace=False)

    cols = 3
    rows = (len(sample_idxs) + cols - 1) // cols
    fig, axes = plt.subplots(rows, cols, figsize=(cols*3, rows*3))
    axes = np.array(axes).flatten()

    for ax_i, idx in enumerate(sample_idxs):
        img_idx   = image_ids[idx].item()
        patch_idx = patch_ids[idx].item()
        row_p     = patch_idx // 28
        col_p     = patch_idx %  28

        img = Image.open(image_paths[img_idx]).convert('RGB').resize((224, 224))
        img_arr = np.array(img)
        # extract 8x8 patch region (scale up for visibility)
        r0, c0 = row_p * 8, col_p * 8
        patch = img_arr[r0:r0+8, c0:c0+8]
        patch_large = np.array(Image.fromarray(patch).resize((64, 64), Image.NEAREST))

        axes[ax_i].imshow(patch_large)
        axes[ax_i].set_title(f'img{img_idx} p{patch_idx}', fontsize=8)
        axes[ax_i].axis('off')

    for ax in axes[len(sample_idxs):]:
        ax.axis('off')

    fig.suptitle(f'Cluster {cluster_id} — Sample Patches', fontsize=13,
fontweight='bold')
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=100, bbox_inches='tight')
    plt.show()
```

4b — Create scripts/validate_clusters.py

```
# scripts/validate_clusters.py — run after clustering to check quality
import sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

import torch, numpy as np, yaml, pickle
from src.parts.patch_clusterer import PatchClusterer, visualise_part_map

cfg = yaml.safe_load(open('configs/unified_config.yaml'))
data = torch.load(cfg['dino']['features_cache'])
n_clusters = cfg['clustering']['n_clusters']

clusterer = PatchClusterer(n_clusters=n_clusters)
labels = clusterer.fit(data['features'])

# Save clusterer and labels
import pickle
pickle.dump(clusterer.kmeans, open('cache/kmeans.pkl', 'wb'))
torch.save(torch.tensor(labels), 'cache/cluster_labels.pt')

# Validate: cluster sizes
```

```
sizes = np.bincount(labels)
print('Cluster sizes:', sizes)
print('Min cluster size:', sizes.min(), '(should be > 50)')

# Visualise part maps for 3 random images
import random
n_images = len(data['image_paths'])
for img_idx in random.sample(range(n_images), min(3, n_images)):
    part_map = clusterer.get_part_map(img_idx, data['image_ids'])
    visualise_part_map(data['image_paths'][img_idx], part_map,
                       n_clusters=n_clusters,
                       save_path=f'cache/part_map_img{img_idx}.png')
print('Done — check cache/part_map_img*.png for visual validation')
```

```
python scripts/validate_clusters.py
```

✅ Validation — Step 4

All cluster sizes > 50 (no empty clusters)

Part map images saved in cache/ — open them and check visually:

- ✓ Different colours appear in different spatial regions
- ✓ Same colour appears in roughly same location across images
- ✗ If every image is one solid colour → clustering failed, increase n_clusters
- ✗ If colours are totally random per-pixel → features not loading correctly

Step 5 — Human Labeling Tool

STEP 5	**Human Labeling Tool** CREATE labeling/label_tool.py · Run it · Save labels.json	⏱ Est. Time **50 min**

This is the human-in-the-loop step. You run this script interactively, view samples from each cluster, and assign semantic labels. Output is labels.json which drives all downstream steps.

5a — Create directory and file

```
mkdir -p labeling
```

5b — Create file: labeling/label_tool.py

```
# labeling/label_tool.py
"""
Interactive cluster labeling tool.
Run with: streamlit run labeling/label_tool.py
Or in Jupyter: from labeling.label_tool import run_jupyter_labeler; run_jupyter_labeler()
"""
import sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

import torch, numpy as np, json, yaml, pickle
from pathlib import Path
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib
matplotlib.use('Agg')
import io, base64


def load_data(config_path='configs/unified_config.yaml'):
    cfg  = yaml.safe_load(open(config_path))
    data = torch.load(cfg['dino']['features_cache'])
    labels_arr = torch.load('cache/cluster_labels.pt').numpy()
    n_clusters = cfg['clustering']['n_clusters']
    return data, labels_arr, n_clusters, cfg


def get_cluster_patches(cluster_id, data, labels_arr, n_samples=9):
    """Return list of (PIL patch image, image_path, patch_idx) for a cluster."""
    idxs = np.where(labels_arr == cluster_id)[0]
    if len(idxs) == 0: return []
    chosen = np.random.choice(idxs, min(n_samples, len(idxs)), replace=False)
    patches = []
    for idx in chosen:
        img_idx   = data['image_ids'][idx].item()
        patch_idx = data['patch_ids'][idx].item()
        r0 = (patch_idx // 28) * 8
        c0 = (patch_idx %  28) * 8
        try:
            img =
Image.open(data['image_paths'][img_idx]).convert('RGB').resize((224,224))
            patch = img.crop((c0, r0, c0+8, r0+8)).resize((80, 80), Image.NEAREST)
            patches.append((patch, data['image_paths'][img_idx], patch_idx))
```

```
        except Exception:
            continue
    return patches


def fig_to_base64(fig):
    buf = io.BytesIO()
    fig.savefig(buf, format='png', bbox_inches='tight', dpi=100)
    buf.seek(0)
    return base64.b64encode(buf.read()).decode()


# ── Streamlit UI ──────────────────────────────────────────────
def run_streamlit():
    import streamlit as st
    st.set_page_config(page_title='Cluster Labeler', layout='wide')
    st.title('🏷️  Human Cluster Labeling Tool')
    st.caption('Label each cluster with the semantic part it represents.')

    data, labels_arr, n_clusters, cfg = load_data()
    labels_path = cfg['concepts']['labels_path']

    # Load existing labels if any
    existing = {}
    if Path(labels_path).exists():
        existing = json.load(open(labels_path))

    st.sidebar.header('Progress')
    labeled_count = sum(1 for v in existing.values() if v.get('label',''))
    st.sidebar.metric('Labeled', f'{labeled_count} / {n_clusters}')

    cluster_id = st.selectbox('Select cluster to label:',
                              list(range(n_clusters)),
                              format_func=lambda x: f'Cluster {x}' + (
                                  f" — {existing.get(str(x),{}).get('label','')}" if
str(x) in existing else ''))

    patches = get_cluster_patches(cluster_id, data, labels_arr, n_samples=9)
    if patches:
        cols = st.columns(3)
        for i, (patch, path, pidx) in enumerate(patches):
            cols[i%3].image(patch, caption=f'img:{Path(path).name[:15]} p:{pidx}',
width=100)

    st.divider()
    current = existing.get(str(cluster_id), {})

    label = st.text_input('Semantic label (e.g. eyes, ears, fur_body, background,
noise):',
                          value=current.get('label', ''))
    include = st.checkbox('Include in classification (uncheck for background/noise)',
                          value=current.get('include', True))
    confidence = st.slider('Confidence (1=unsure, 3=certain)', 1, 3,
                           value=current.get('confidence', 2))
    notes = st.text_area('Notes (optional):', value=current.get('notes', ''), height=60)

    if st.button('💾  Save label', type='primary'):
        existing[str(cluster_id)] = {
            'label': label.strip(),
            'include': include,
```

```
            'confidence': confidence,
            'notes': notes.strip()
        }
        Path(labels_path).parent.mkdir(parents=True, exist_ok=True)
        json.dump(existing, open(labels_path, 'w'), indent=2)
        st.success(f'Saved: Cluster {cluster_id} → "{label}"')
        st.rerun()

    if labeled_count == n_clusters:
        st.balloons()
        st.success('All clusters labeled! You can now run Step 6.')
        st.json(existing)


if __name__ == '__main__':
    run_streamlit()
```

5c — Run the labeling tool

```
streamlit run labeling/label_tool.py
# Opens at http://localhost:8501
# Go through all 8 clusters, assign labels, click Save after each
# When done, cache/labels.json will contain all your labels
```

✅ Validation — Step 5

Run: python -c "import json; d=json.load(open('cache/labels.json')); print(json.dumps(d, indent=2))"
Expected: all 8 cluster IDs present, each with label, include, confidence
At least 4 clusters should have include: true (these become your concepts)
Example good output:
{"0": {"label": "eyes", "include": true, "confidence": 3}, ...}

Step 6 — Concept Vector Builder

STEP 6	**Concept Vector Builder** CREATE src/concepts/concept_builder.py	⏱ Est. Time **40 min**

6a — Create directory and file

```
mkdir -p src/concepts
```

6b — Create file: src/concepts/concept_builder.py

```
# src/concepts/concept_builder.py
import torch, json, yaml
import torch.nn.functional as F
import numpy as np
from pathlib import Path


def build_concept_vectors(config_path: str = 'configs/unified_config.yaml') -> dict:
    """
    For each labeled, included cluster: compute the mean DINO feature vector.
    This mean vector is the concept vector — it represents what that
    semantic part looks like in DINO feature space.
    Returns: dict of {label: tensor[384]}
    """
    cfg = yaml.safe_load(open(config_path))

    data        = torch.load(cfg['dino']['features_cache'])
    labels_arr  = torch.load('cache/cluster_labels.pt').numpy()
    human_labels= json.load(open(cfg['concepts']['labels_path']))

    features = data['features']   # [N, 384]

    concept_vectors = {}
    concept_meta    = {}

    print('Building concept vectors:')
    for cluster_id_str, meta in human_labels.items():
        cluster_id = int(cluster_id_str)
        label      = meta.get('label', '').strip()
        include    = meta.get('include', True)

        if not label or not include:
            print(f'  Cluster {cluster_id}: "{label}" — EXCLUDED')
            continue

        mask = (labels_arr == cluster_id)
        if mask.sum() == 0:
            print(f'  Cluster {cluster_id}: empty — skipped')
            continue

        cluster_feats = features[mask]                  # [M, 384]
        mean_vec      = cluster_feats.mean(dim=0)       # [384]
        # L2 normalise the concept vector
        mean_vec      = F.normalize(mean_vec, dim=0)
```

```
        concept_vectors[label] = mean_vec
        concept_meta[label]    = {
            'cluster_id':  cluster_id,
            'n_patches':   int(mask.sum()),
            'confidence':  meta.get('confidence', 2),
            'notes':       meta.get('notes', '')
        }
        print(f'  Cluster {cluster_id}: "{label}" — {mask.sum():,} patches')

    # Save
    vectors_path = cfg['concepts']['vectors_cache']
    Path(vectors_path).parent.mkdir(parents=True, exist_ok=True)
    torch.save({'vectors': concept_vectors, 'meta': concept_meta}, vectors_path)
    print(f'Saved {len(concept_vectors)} concept vectors → {vectors_path}')
    return concept_vectors


def compute_concept_scores_all(config_path: str = 'configs/unified_config.yaml'):
    """
    Pre-compute concept activation scores for every image.
    Score = max cosine similarity across all 784 patches.
    Saves scores matrix [N_images, N_concepts] to cache.
    """
    cfg = yaml.safe_load(open(config_path))
    data        = torch.load(cfg['dino']['features_cache'])
    saved       = torch.load(cfg['concepts']['vectors_cache'])
    concept_vectors = saved['vectors']

    image_paths  = data['image_paths']
    image_ids    = data['image_ids']
    features     = data['features']
    image_labels = data['image_labels']
    concept_names= list(concept_vectors.keys())
    n_images     = len(image_paths)
    n_concepts   = len(concept_names)

    scores_matrix = torch.zeros(n_images, n_concepts)   # [N_images, N_concepts]

    print(f'Computing concept scores for {n_images} images × {n_concepts} concepts...')
    for img_idx in range(n_images):
        mask      = (image_ids == img_idx)
        img_feats = features[mask]                              # [784, 384]
        img_norm  = F.normalize(img_feats, dim=-1)             # [784, 384]

        for c_idx, c_name in enumerate(concept_names):
            vec      = F.normalize(concept_vectors[c_name], dim=0)   # [384]
            sims     = (img_norm @ vec)                              # [784]
            scores_matrix[img_idx, c_idx] = sims.max()            # peak activation

    scores_cache = cfg['concepts']['scores_cache']
    torch.save({
        'scores':        scores_matrix,
        'concept_names': concept_names,
        'image_labels':  image_labels,
        'class_names':   data['class_names'],
    }, scores_cache)
    print(f'Saved concept scores {scores_matrix.shape} → {scores_cache}')
    return scores_matrix, concept_names
```

```
if __name__ == '__main__':
    build_concept_vectors()
    compute_concept_scores_all()
```

```
python src/concepts/concept_builder.py
```

✅ Validation — Step 6

Expected output: list of included concepts with patch counts

Run: python -c "import torch; d=torch.load('cache/concept_vectors.pt'); print(list(d['vectors'].keys())); print('scores:', torch.load('cache/concept_scores.pt')['scores'].shape)"

Expected: list of your concept names (e.g. ['eyes', 'ears', 'fur_body', ...])

Expected: scores shape [N_images, N_concepts]

Each score should be between 0.0 and 1.0 (cosine similarity)

Step 7 — Concept Classifier & Explanation Renderer

STEP 7	**Concept Classifier + Explanations** CREATE src/classification/concept_classifier.py	⏱ Est. Time **60 min**

7a — Create directory and file

```
mkdir -p src/classification
```

7b — Create file: src/classification/concept_classifier.py

```
# src/classification/concept_classifier.py
import torch, yaml, json
import numpy as np
import torch.nn.functional as F
import matplotlib.pyplot as plt
import matplotlib
matplotlib.use('Agg')
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import StandardScaler
from pathlib import Path
import pickle


class ConceptClassifier:
    """
    Linear probe classifier on top of concept activation scores.
    Intentionally simple — interpretability requires linearity.
    """

    def __init__(self, C: float = 1.0, max_iter: int = 1000, random_state: int = 42):
        self.clf     = LogisticRegression(C=C, max_iter=max_iter,
                                          random_state=random_state, multi_class='auto')
        self.scaler  = StandardScaler()
        self.concept_names = None
        self.class_names   = None

    def fit(self, scores: torch.Tensor, labels: list,
            concept_names: list, class_names: list,
            test_size: float = 0.2):
        """
        scores: [N_images, N_concepts]
        labels: [N_images] integer class ids
        """
        self.concept_names = concept_names
        self.class_names   = class_names

        X = scores.numpy() if isinstance(scores, torch.Tensor) else scores
        y = np.array(labels)

        X_tr, X_te, y_tr, y_te = train_test_split(X, y,
                                                  test_size=test_size,
                                                  random_state=42,
```

```
                                                        stratify=y if len(np.unique(y)) > 1
else None)

        X_tr = self.scaler.fit_transform(X_tr)
        X_te = self.scaler.transform(X_te)

        self.clf.fit(X_tr, y_tr)
        y_pred = self.clf.predict(X_te)

        acc = accuracy_score(y_te, y_pred)
        print(f'\nClassification Accuracy: {acc:.3f}')
        print(classification_report(y_te, y_pred,
                                    target_names=[class_names[i] for i in
np.unique(y_te)]))

        self._print_concept_weights()
        return acc

    def _print_concept_weights(self):
        print('\nConcept weights per class:')
        for i, cname in enumerate(self.class_names):
            if i >= len(self.clf.coef_): continue
            weights = self.clf.coef_[i] if len(self.clf.coef_) > 1 else self.clf.coef_[0]
            sorted_idx = np.argsort(np.abs(weights))[::-1]
            print(f'  Class "{cname}":')
            for j in sorted_idx[:5]:
                print(f'    {self.concept_names[j]:20s} weight={weights[j]:+.3f}')

    def predict_with_explanation(self, image_features: torch.Tensor,
                                 concept_vectors: dict) -> dict:
        """
        Given DINO features for ONE image [784, 384],
        returns prediction + full concept breakdown.
        """
        # Compute concept scores for this image
        scores = {}
        img_norm = F.normalize(image_features, dim=-1)   # [784, 384]
        for c_name, vec in concept_vectors.items():
            v_norm = F.normalize(vec, dim=0)             # [384]
            sims   = (img_norm @ v_norm)                 # [784]
            scores[c_name] = sims.max().item()

        score_vec = np.array([[scores[c] for c in self.concept_names]])
        score_vec_scaled = self.scaler.transform(score_vec)

        pred_class_id  = self.clf.predict(score_vec_scaled)[0]
        pred_proba     = self.clf.predict_proba(score_vec_scaled)[0]
        pred_class_name= self.class_names[pred_class_id]
        confidence     = pred_proba.max()

        # Concept contributions = score × coefficient
        coef_idx = pred_class_id if len(self.clf.coef_) > 1 else 0
        weights  = self.clf.coef_[coef_idx]
        contributions = {c: scores[c] * weights[i]
                         for i, c in enumerate(self.concept_names)}

        return {
            'prediction':    pred_class_name,
            'confidence':    float(confidence),
            'concept_scores':  scores,
```

```
            'contributions':   contributions,
            'class_proba':     {self.class_names[i]: float(p) for i, p in
enumerate(pred_proba)},
        }

    def save(self, path: str):
        payload = {
            'clf': self.clf, 'scaler': self.scaler,
            'concept_names': self.concept_names,
            'class_names': self.class_names
        }
        pickle.dump(payload, open(path, 'wb'))
        print(f'Classifier saved → {path}')

    @classmethod
    def load(cls, path: str):
        payload = pickle.load(open(path, 'rb'))
        obj = cls()
        obj.clf = payload['clf']; obj.scaler = payload['scaler']
        obj.concept_names = payload['concept_names']
        obj.class_names   = payload['class_names']
        return obj


def render_explanation(result: dict, save_path: str = None):
    """
    Render a horizontal bar chart showing concept contributions.
    This is the explainability visualisation for your dissertation.
    """
    contribs  = result['contributions']
    concepts  = list(contribs.keys())
    values    = [contribs[c] for c in concepts]
    scores    = [result['concept_scores'][c] for c in concepts]

    sorted_idx = sorted(range(len(values)), key=lambda i: abs(values[i]), reverse=True)
    concepts   = [concepts[i] for i in sorted_idx]
    values     = [values[i]   for i in sorted_idx]
    scores     = [scores[i]   for i in sorted_idx]

    colors = ['#2196F3' if v >= 0 else '#F44336' for v in values]

    fig, ax = plt.subplots(figsize=(9, max(3, len(concepts)*0.6 + 1)))
    bars = ax.barh(concepts, values, color=colors, edgecolor='white', height=0.6)

    for i, (bar, score) in enumerate(zip(bars, scores)):
        ax.text(bar.get_width() + 0.005, bar.get_y() + bar.get_height()/2,
                f'act={score:.2f}', va='center', ha='left', fontsize=9, color='#555')

    ax.set_xlabel('Concept Contribution (activation × weight)', fontsize=10)
    ax.set_title(f"Prediction: {result['prediction'].upper()}  ""," 
                 f"confidence={result['confidence']:.0%}", fontsize=12,
fontweight='bold')
    ax.axvline(0, color='grey', linewidth=0.8)
    ax.spines[['top','right']].set_visible(False)
    plt.tight_layout()

    if save_path:
        plt.savefig(save_path, dpi=120, bbox_inches='tight')
        print(f'Explanation saved → {save_path}')
    plt.show()
```

```
    return fig
```

✅ **Validation — Step 7**
No standalone run yet — this module is called by Step 8.
Syntax check: python -c "from src.classification.concept_classifier import ConceptClassifier; print('ok')"
Expected: ok

Step 8 — End-to-End Runner with W&B Logging

STEP 8	**End-to-End Runner + W&B Logging** CREATE experiments/run_dino_pipeline.py — ties all stages together	⏱ Est. Time **60 min**

8a — Create file: experiments/run_dino_pipeline.py

```
# experiments/run_dino_pipeline.py
"""
End-to-end DINO parts discovery pipeline.
Runs all stages in sequence, logs to W&B.

Usage:
  python experiments/run_dino_pipeline.py --stage all
  python experiments/run_dino_pipeline.py --stage cluster
  python experiments/run_dino_pipeline.py --stage classify
  python experiments/run_dino_pipeline.py --stage explain --image path/to/image.jpg
"""
import sys, os, argparse
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

import torch, yaml, json, pickle
import numpy as np
from pathlib import Path


def stage_extract(cfg):
    print('\n' + '='*60)
    print('STAGE 1+2: Feature Extraction + Caching')
    print('='*60)
    from experiments.extract_dino_features import extract_all
    return extract_all()


def stage_cluster(cfg):
    print('\n' + '='*60)
    print('STAGE 3: Patch Clustering')
    print('='*60)
    from src.parts.patch_clusterer import PatchClusterer
    data = torch.load(cfg['dino']['features_cache'])
    n_clusters = cfg['clustering']['n_clusters']
    clusterer = PatchClusterer(n_clusters=n_clusters,
                               random_seed=cfg['clustering']['random_seed'])
    labels = clusterer.fit(data['features'])
    torch.save(torch.tensor(labels), 'cache/cluster_labels.pt')
    clusterer.save('cache/kmeans.pkl')
    return clusterer, labels


def stage_concepts(cfg):
    print('\n' + '='*60)
    print('STAGE 4+5: Build Concept Vectors + Compute Scores')
    print('='*60)
    from src.concepts.concept_builder import build_concept_vectors,
compute_concept_scores_all
    vectors  = build_concept_vectors()
    scores, concept_names = compute_concept_scores_all()
```

```
    return vectors, scores, concept_names


def stage_classify(cfg):
    print('\n' + '='*60)
    print('STAGE 6: Concept Classification')
    print('='*60)
    from src.classification.concept_classifier import ConceptClassifier

    scores_data   = torch.load(cfg['concepts']['scores_cache'])
    scores        = scores_data['scores']
    concept_names = scores_data['concept_names']
    image_labels  = scores_data['image_labels']
    class_names   = scores_data['class_names']

    clf = ConceptClassifier(
        C=cfg['classification']['C'],
        max_iter=cfg['classification']['max_iter'],
        random_state=cfg['classification']['random_seed']
    )
    acc = clf.fit(scores, image_labels, concept_names, class_names,
                  test_size=cfg['classification']['test_size'])
    clf.save('cache/concept_classifier.pkl')
    return clf, acc


def stage_explain(cfg, image_path: str):
    print('\n' + '='*60)
    print(f'STAGE 7: Explain prediction for {image_path}')
    print('='*60)
    from src.models.dino_extractor import DinoExtractor
    from src.classification.concept_classifier import ConceptClassifier,
render_explanation

    extractor = DinoExtractor(model_name=cfg['dino']['model'],
                              device=cfg['dino']['device'],
                              image_size=cfg['dino']['image_size'])
    clf      = ConceptClassifier.load('cache/concept_classifier.pkl')
    saved    = torch.load(cfg['concepts']['vectors_cache'])
    vectors  = saved['vectors']

    img_feats = extractor.extract_from_path(image_path)   # [784, 384]
    result    = clf.predict_with_explanation(img_feats, vectors)

    print(f"\nPrediction : {result['prediction']}")
    print(f"Confidence : {result['confidence']:.2%}")
    print('\nConcept Activations:')
    for c, score in sorted(result['concept_scores'].items(),
                           key=lambda x: x[1], reverse=True):
        bar = '▇' * int(score * 20)
        print(f'  {c:20s} {score:.3f}  {bar}')

    save_path = f'cache/explanation_{Path(image_path).stem}.png'
    render_explanation(result, save_path=save_path)
    return result


def log_to_wandb(cfg, acc, concept_names):
    """Log results to W&B — reuses your existing W&B setup."""
    try:
```

```
        import wandb
        wandb.init(project=cfg.get('wandb', {}).get('project', 'dino-parts-discovery'),
                   name='dino-concept-pipeline',
                   config={
                       'model':       cfg['dino']['model'],
                       'n_clusters':  cfg['clustering']['n_clusters'],
                       'n_concepts':  len(concept_names),
                       'clf_C':       cfg['classification']['C'],
                   })
        wandb.log({'test_accuracy': acc, 'n_concepts': len(concept_names)})

        # Log part map images if they exist
        for img_path in Path('cache').glob('part_map_img*.png'):
            wandb.log({f'part_maps/{img_path.name}': wandb.Image(str(img_path))})
        for img_path in Path('cache').glob('explanation_*.png'):
            wandb.log({f'explanations/{img_path.name}': wandb.Image(str(img_path))})

        wandb.finish()
        print('W&B logging complete.')
    except Exception as e:
        print(f'W&B logging skipped: {e}')


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--stage', default='all',
                        choices=['all', 'extract', 'cluster', 'concepts', 'classify',
'explain'])
    parser.add_argument('--image', default=None, help='Image path for explain stage')
    parser.add_argument('--config', default='configs/unified_config.yaml')
    args = parser.parse_args()

    cfg = yaml.safe_load(open(args.config))

    if args.stage in ('all', 'extract'):
        stage_extract(cfg)

    if args.stage in ('all', 'cluster'):
        stage_cluster(cfg)

    if args.stage in ('all', 'concepts'):
        # Requires cache/labels.json — run labeling tool first!
        if not Path(cfg['concepts']['labels_path']).exists():
            print('ERROR: labels.json not found. Run labeling tool first:')
            print('  streamlit run labeling/label_tool.py')
            sys.exit(1)
        vectors, scores, concept_names = stage_concepts(cfg)

    if args.stage in ('all', 'classify'):
        clf, acc = stage_classify(cfg)
        scores_data = torch.load(cfg['concepts']['scores_cache'])
        log_to_wandb(cfg, acc, scores_data['concept_names'])

    if args.stage == 'explain':
        if not args.image:
            print('ERROR: --image required for explain stage')
            sys.exit(1)
        stage_explain(cfg, args.image)

    print('\n✅ Pipeline complete!')
```

```
if __name__ == '__main__':
    main()
```

8b — Run the complete pipeline

Run stages in this exact order. Do not skip stages.

```
# Stage 1: Extract features (one-time, ~15-20 min)
python experiments/run_dino_pipeline.py --stage extract

# Stage 2: Cluster patches (~2 min)
python experiments/run_dino_pipeline.py --stage cluster

# Stage 3: Validate clusters visually (look at cache/part_map_img*.png)
python scripts/validate_clusters.py

# Stage 4: Label clusters (interactive — opens browser)
streamlit run labeling/label_tool.py

# Stage 5: Build concept vectors + scores (~5 min)
python experiments/run_dino_pipeline.py --stage concepts

# Stage 6: Train classifier + log to W&B
python experiments/run_dino_pipeline.py --stage classify

# Stage 7: Explain a single image
python experiments/run_dino_pipeline.py --stage explain --image data/cat/any_cat.jpg
```

✅ Final Validation — Step 8 (Definition of Done)

✓ cache/dino_features.pt exists and loads
✓ cache/cluster_labels.pt exists, shape [N_patches]
✓ cache/part_map_img*.png — part maps show spatial regions, not noise
✓ cache/labels.json — all clusters labeled, at least 4 included
✓ cache/concept_vectors.pt — vectors saved, correct concept names
✓ cache/concept_scores.pt — shape [N_images, N_concepts]
✓ cache/concept_classifier.pkl — classifier saved
✓ Accuracy printed to console (any result > chance level is a success for day 1)
✓ cache/explanation_*.png — bar chart showing concept contributions
✓ W&B run visible at wandb.ai with accuracy + part map images logged

Quick Reference — Files Changed Summary

Action	File / Location	Detail
MODIFY	requirements.txt	`Add 6 lines at bottom (Step 1b)`
MODIFY	configs/unified_config.yaml	`Add dino/clustering/concepts/classification blocks at bottom (Step 1c)`
CREATE	cache/ (directory)	`mkdir -p cache (Step 1d)`
CREATE	src/models/dino_extractor.py	`DinoExtractor class — frozen DINO ViT (Step 2)`
CREATE	experiments/extract_dino_features.py	`One-time feature extraction to cache (Step 3)`
CREATE	scripts/validate_clusters.py	`Visual validation of part maps (Step 4)`
CREATE	src/parts/patch_clusterer.py	`PatchClusterer + visualise helpers (Step 4)`
CREATE	labeling/ (directory)	`mkdir -p labeling (Step 5)`
CREATE	labeling/label_tool.py	`Streamlit human labeling UI (Step 5)`
CREATE	src/concepts/concept_builder.py	`Concept vectors + score matrix (Step 6)`
CREATE	src/classification/concept_classifier.py	`Linear probe + explanation renderer (Step 7)`
CREATE	experiments/run_dino_pipeline.py	`End-to-end runner with W&B logging (Step 8)`
ARCHIVE	src/training/part_trainer.py	`Do not delete — just do not run`
ARCHIVE	src/models/slot_attention.py	`Do not delete — just do not run`
ARCHIVE	experiments/train_part_discovery.py	`Do not delete — just do not run`

If something breaks — debug checklist

MPS errors → add device='cpu' fallback in dino_extractor.py __init__
ImportError on src.models → ensure you are running from project root
Empty clusters → reduce n_clusters in config (try 6 instead of 8)
labels.json missing → run streamlit labeling tool before concepts stage
Low accuracy → expected with 1 class; add dog images for meaningful classification
W&B errors → pipeline still works, W&B is optional logging only